

Code-Layout, Readability and Reusability

Date	30/06/2026
Team ID	xxxx
Project Name	Personalized Networking Assistant
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability

This document evaluates the quality of the Personalized Networking Assistant codebase in terms of structure, readability, and reusability.

Code Layout Checklist

S. No	Code Quality Parameter	Description	Followed	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	PEP8-style 4-space indentation throughout backend and frontend files
2	Proper File Structure	Files and folders are logically organized	Yes	backend/, data/, tests/ separated; frontend.py at project root
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	e.g. event_context, extracted_themes, starter_list, fact_check_result
4	Function / Method Names	Functions are descriptively named	Yes	e.g. analyze_event(), generate_starters(), verify_fact(), save_history()
5	Code Comments	Inline and block comments explain logic	Yes	Docstrings and inline comments explain Gemini prompt construction and fallback logic
6	Modular Design	Code is split into reusable functions/modules	Yes	Split into event_analyzer.py, topic_generator.py, fact_checker.py, main.py
7	No Redundant Code	Duplicate or unused code is removed	Yes	Shared logic (Gemini client setup) centralized instead of duplicated per module
8	Error Handling	Exceptions and errors are handled gracefully	Yes	try/except around Gemini API calls with fallback templates on quota/error failures

Reusable Components / Modules

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level
1	event_analyzer.py	Python	Called by main.py's /analyze-event endpoint	High
2	topic_generator.py	Python	Called by main.py's /generate-starters endpoint, includes fallback templates	High
3	fact_checker.py	Python	Called by main.py's /fact-check endpoint	High

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level
4	history.json save/load helpers	Python	Used by both starter generation and history-view flows	Medium

Overall Code Quality Assessment

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5	Clear backend/frontend split, consistent structure
Readability	5	Descriptive names and comments make logic easy to follow
Reusability	4	Core services are modular; could extract a shared Gemini-client helper for even more reuse
Documentation / Comments	5	README covers setup; docstrings/comments explain key functions
Overall Score	4.75 / 5	Well-structured, tested (15 passing pytest tests), and maintainable codebase